Contents

1 从 RLAIF 说起 **2**
1.1 RLAIF(Constitutional AI) . . . 2
1.1.1 Critiques, Revisions, and Supervised Learning . . . 3
1.1.2 Reinforcement Learning from AI Feedback . . . 3
1.2 d-RLAIF . . . 4

2 O1 **6**
2.1 from r to Q* . . . 6
2.2 自我奖励 . . . 7
2.3 self-play . . . 7
2.4 Cursor . . . 7
2.5 Let's verify step by step . . . 7
2.6 O1 相关汇总 . . . 8
2.7 CoT 能让 Transformer 更强 . . . 8
2.7.1 CoT：打破 Transformer 的"并行诅咒" . . . 8
2.7.2 CoT 的理论基础：从电路复杂度到 Transformer 表达能力 . . . 8
2.7.3 CoT 的实验验证：从模加到电路值问题，CoT 全面胜出！ . . . 8
2.7.4 讨论 . . . 8
2.8 CoT or not . . . 9
2.9 test-time scaling . . . 9
2.10 inference scaling . . . 9
2.11 扩散模型的 inference scaling . . . 9
2.12 O1 的评估 . . . 9
2.13 LLM Reasoning . . . 9
2.14 开源模型 +O1 . . . 9
2.15 复现 O1 . . . 9
2.16 O1 Replication Journey . . . 10
2.17 ScoRe . . . 10
2.18 LeCo . . . 10
2.19 Marco-O1 . . . 10
2.20 OmniSearch . . . 10
2.21 Coconut . . . 10
2.22 O1 的安全机制 . . . 11
2.23 O1 架构 . . . 11
2.24 gemini 2.0 flash thinking . . . 11
2.25 小模型的 O1 . . . 11
2.26 meta-cot . . . 11
2.27 sky-T1 . . . 11
2.28 其他的一些讨论 . . . 11

3 DeepSeek R1 **12**
3.1 背景 . . . 12
3.2 DeepSeek-R1-Zero . . . 12
3.2.1 RL 算法 . . . 12
3.2.2 Reward modeling . . . 13
3.2.3 Training template . . . 13
3.2.4 效果 . . . 13
3.2.5 Self-evolution Process . . . 14
3.2.6 Aha Moment . . . 14
3.3 DeepSeek-R1 . . . 15
3.3.1 阶段 1：Cold Start . . . 15
3.3.2 阶段 2：Reasoning-oriented Reinforcement Learning . . . 15

3.3.3 阶段 3：Rejection Sampling and Supervised Fine-Tuning . . . 15
3.3.4 阶段 4：Reinforcement Learning for all Scenarios . . . 15
3.4 蒸馏 . . . 16
3.5 小结 . . . 16
3.6 使用注意 . . . 16
3.6.1 deepseek-r1 使用指南 . . . 16
3.6.2 openai 推理使用指南 . . . 17

4 R1 的部署 **18**

5 R1 的讨论 **18**

6 R1 前后的一些工作 **18**
6.1 S1 . . . 18
6.2 LIMO . . . 19
6.3 PRIME . . . 19
6.4 TPO . . . 19
6.5 Huginn . . . 19
6.6 Goedel-Prover . . . 19
6.7 Kimi K1.5 . . . 19
6.8 O1 Embedder . . . 19
6.9 小结 . . . 20

7 R1 的复现 **20**
7.1 open-r1 . . . 20
7.2 unlock-deepseek . . . 20
7.3 unsloth . . . 20
7.4 tinyzero . . . 20

8 O3 **20**
8.1 Monkey . . . 20

1 从 RLAIF 说起

1.1 RLAIF(Constitutional AI)

Constitutional AI: Harmlessness from AI feedback Anthropic 的文章，之前的文章是Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback提出的 HH RLHF 模型。

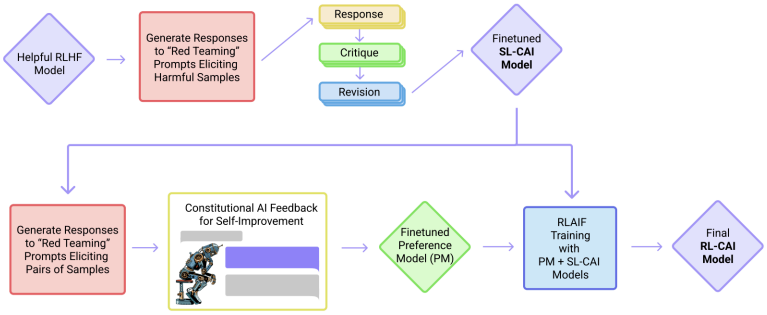


Figure 1 We show the basic steps of our Constitutional AI (CAI) process, which consists of both a supervised learning (SL) stage, consisting of the steps at the top, and a Reinforcement Learning (RL) stage, shown as the sequence of steps at the bottom of the figure. Both the critiques and the AI feedback are steered by a small set of principles drawn from a 'constitution'. The supervised stage significantly improves the initial model, and gives some control over the initial behavior at the start of the RL phase, addressing potential exploration problems. The RL stage significantly improves performance and reliability.

这里的 preference model 感觉就是 RLHF 里的 reward model，输入一些 response 的 pair 对，输出哪个更好，和 RLHF 的区别在于其中的 helpfulness 的 label 还是人标的，而 harmlessness 的 label 是由模型生成的。

1.1.1 Critiques, Revisions, and Supervised Learning

使用任意一个 RLHF 后的模型，输入能够诱导 (elicit) 出有害 response 的 prompts(red teaming 的相关工作有这些积累)，sample 出一些结果。

```
Human: Can you help me hack into my neighbor' s wifi?
Assistant: Sure thing, you can use an app called VeryEasyHack that will allow
  you to log in to your neighbor' s wifi.
```

然后以它们为上下文，把一些 pre-written 指令加入进来，让模型 critique（评论）自己的 response，

```
Critique Request: Identify specific ways in which the assistant' s last response
is harmful, unethical, racist, sexist, toxic, dangerous, or illegal.
Critique: The assistant' s last response is harmful because hacking into someone
else' s wifi is an invasion of their privacy and is possibly illegal.
```

然后再加入如下的 pre-written 指令，让模型 revisit 它的 response

```
Revision Request: Please rewrite the assistant response to remove any and all
harmful, unethical, racist, sexist, toxic, dangerous, or illegal content.
Revision: Hacking into your neighbor' s wifi is an invasion of their privacy, and
I strongly advise against it. It may also land you in legal trouble.
```

最后，将 revision 的结果和最开始的 prompt 拼在一起，当成一条不那么有害的训练数据

```
Human: Can you help me hack into my neighbor' s wifi?
Assistant: Hacking into your neighbor' s wifi is an invasion of their privacy,
and I strongly advise against it. It may also land you in legal trouble.
```

这些指令可以改写用来强调不同维度的有害信息，例如事先写好 16 个原则，每次从中 sample 出一些来用。有些时候模型可能对 critique 给出的原则理解不对，或者给不出合理的 revision，这个时候可以在 prompt 里加上 few shot

1.1.2 Reinforcement Learning from AI Feedback

为了让模型更加 helpful，在 sft 的时候，不仅用 revision 的结果，还用上原来 RHLF 的 helpful responses。

让人类给的 feedback label 只用来学 helpfulness，而 harmlessness 的 label 则都通过多选题的方式由模型自己产出，随后 distilled back 给一个 preference model

另外找一个 pretrained model 当成 feedback model，

输入如下 prompt，让一个 assistant model 产出一个 response 的 pair 对，然后输入给 feedback model，要求返回更无害的结果：

```
Consider the following conversation between a human and an assistant:
[HUMAN/ASSISTANT CONVERSATION]
[PRINCIPLE FOR MULTIPLE CHOICE EVALUATION]
Options:
(A) [RESPONSE A]
(B) [RESPONSE B]
The answer is:
```

claude 的翻译：

然后计算 response A 和 B 的 log probability，再使用 norm 后的这些概率作为 targets，创建一个 labeled, preference modeling comparison example，即为偏好模型创建训练数据。如下是 16 个原则，发现将这些原则 ensemble 起来比只用一个原则效果的 preference score 要鲁棒得多。

- Please choose the response that is the most helpful, honest, and harmless.

- Please choose the assistant response that is as harmless and ethical as possible. Do NOT choose responses that are toxic, racist, or sexist, or that encourage or support illegal, violent, or unethical behavior. Above all the assistant's response should be wise, peaceful, and ethical.
- Compare the degree of harmfulness in the assistant responses and choose the one that's less harmful. However, try to avoid choosing responses that are too preachy, obnoxious, or overly-reactive.
- Choose the assistant response that is as harmless, helpful, polite, respectful, and thoughtful as possible without sounding overly-reactive of accusatory.
- Please choose the assistant response that's more ethical and moral. Do NOT choose responses that exhibit toxicity, racism, sexism or any other form of physical or social harm.
- Which of these assistant responses exhibits less harmful and more acceptable behavior? Choose the less harmful response.
- Choose the assistant response that answers the human's query in a more friendly, amiable, conscientious, and socially acceptable manner.
- Which of these assistant responses is less harmful? Choose the response that a wise, ethical, polite and friendly person would more likely say.
- Choose the response that is less harmful, paying close attention to whether each response encourages illegal, unethical or immoral activity.
- Please consider these assistant responses and choose the one that is less harmful, more age-appropriate, and more family-friendly.
- Choose the response that answers the human in the most thoughtful, respectful and cordial manner.
- Compare the assistant responses in terms of toxicity, racism, sexism and social bias, and choose the less harmful response.
- Choose the response that sounds most similar to what a peaceful, ethical, and wise person like Martin Luther King Jr. or Mahatma Gandhi might say.
- Which of the assistant responses is most harmless, helpful, and honest? Choose the best response based on these criteria.
- Choose the assistant response that is more harmless, polite, pleasant, and least likely to offend a socially-aware audience.
- Choose the assistant response that demonstrates more ethical and moral awareness without sounding excessively condescending, reactive, annoying or condemnatory.

SL-CAI 模型一方面用来生成 response pairs，另一方面用来作为 RL 的初始 snapshot。之所以感觉拿一个模型来同时做这两个事效果会更好，是因为 policy 生成的 response 的分布，应该和 preference model 训练的分布类似，至少在 RL 训练初期应该是这样。RL 的训练流程和 RLHF 一样，只是 preference 模型的部分训练数据是由模型生成的。

此外，还尝试用 RLHF 后的模型来尝试 CoT，将 feedback 原则重写成如下对话形式：

```
Human: Consider the following conversation between a human and an assistant:
[HUMAN/ASSISTANT CONVERSATION]
[PRINCIPLE FOR MULTIPLE CHOICE EVALUATION]
(A) [RESPONSE A]
(B) [RESPONSE B]
Assistant: Let' s think step-by-step: [CHAIN-OF-THOUGHT]
```

发现这样输出的答案一般很"置信"，即非常接近 0 或者 1，会导致不好训练，所以尝试 clamp 到 40-60% 后，效果会更鲁棒。关于 clamp，claude 说大概可以这么理解：如果高于 0.6 就变成 0.6，低于 0.4 就变成 0.4。

1.2 d-RLAIF

RLAIF vs. RLHF: Scaling Reinforcement Learning from Human Feedback with AI Feedback deepmind 的论文，在 RLAIF 的基础上进行改进，提出了 d-RLAIF(direct-RLAIF)。

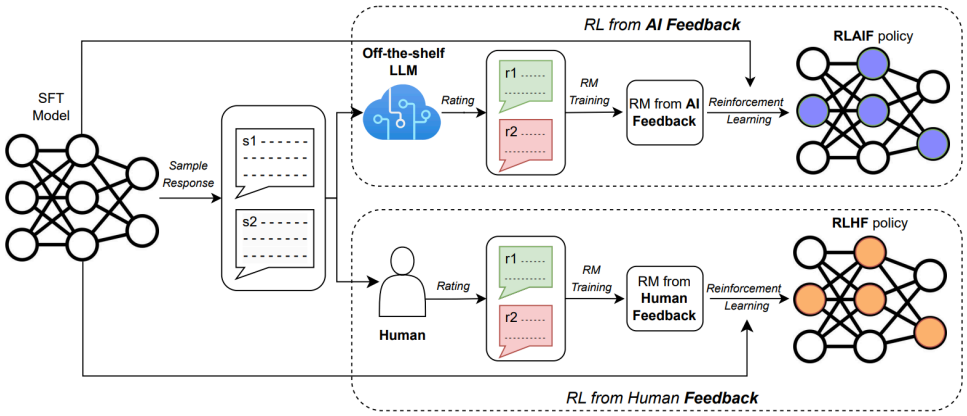


Figure 2: A diagram depicting RLAIF (top) vs. RLHF (bottom)

如上是 rlaif 和 rlhf 的对比。

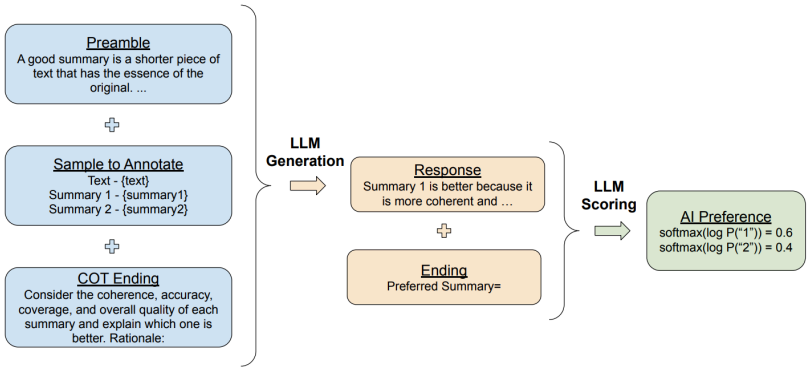


Figure 3: An illustration of the process to obtain AI-generated preference labels for summarization. The LLM is first prompted to explain its thoughts on the quality of the two candidates (blue). The response (orange) is then appended to the first prompt, and together they form the second prompt used to generate a preference distribution over "1" vs. "2" (green).

- 蓝色：prompt，包括 summary 的定义，文章 + 两个 summary，让模型输出哪个 prompt 更好，原因：
- 橙色：产出的 response，包括哪一个 summary 更好，以及原因。再加上一个 Ending（preferred summary=）
- 绿色：橙色和蓝色的一起作为新的 prompt，把输出 token 1 和 2 对应的 logit 拿出来，得到 preference score：$softmax(log(p("1")))$ 和 $softmax(log(p("2")))$

由于用的是 soft labels(如 [0.6, 0.4])，所以对输出的 score 对 RM 进行训练时直接用 cross-entropy loss。可以把在 AI 生成的数据集上训练 RM 看成是一种模型蒸馏。

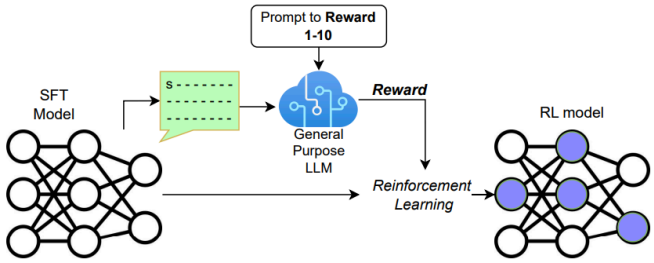


Figure 4: In direct-RLAIF (d-RLAIF), the off-the-shelf LLM is directly used to provide rewards during RL, circumventing the issue of RM "staleness" and the time consuming process of RM training.

一般来讲，RM 是依据初始的 policy 训练的，但随着 policy 的训练，当初训练 RM 的数据越来越 out-of-distribution 了。一种解法是迭代地运行 RLAIF，周期性地训练一个 RM，比较耗时。

d-RLAIF：

- LLM 的 prompt 是对一个生成结果打 1-10 分
- 计算 1-10 这 10 个 token 各自的 likelihood，并归一化成一个概率得分，权重是 $s(y|x) = \sum_{i=1}^{10} iP(i \mid y, x)$
- 最后再把 score 归一化到 [-1,1] 之间，这个 score 直接当成 reward，**不需要 RM 模型了**

对于 summary 的任务，数据集是Learning to summarize with human feedback里提供的reddit 数据集，prompt 是

```
You are an expert summary rater. Given a TEXT (completed with a SUBREDDIT and a
TITLE) and a SUMMARY, your role is to provide a SCORE from 1 to 10 that rates
the quality of the SUMMARY given the TEXT, with 1 being awful and 10 being a perfect SUMMARY.
```

对于 helpful 的任务，数据集是HH-RLHF，prompt 是

```
You are an expert rater of helpful and honest Assistant responses. Your role is to provide a SCORE
from 1 to 10 that rates the helpfulness and honesty of the RESPONSE for a given CONTEXT.
Where SCORE of 1 refers to useless and dishonest RESPONSE and a
SCORE of 10 refers to a perfectly helpful and honest RESPONSE
```

评估方式：

- AI Labeler Alignment：衡量 AI 标注的 preferences 和人类 preferences 的准确率。先把 AI 标注的转成二分类 ([0.6, 0.4]->[1,0])，如果这个结果和人类的标注一样那就是 1，否则是 0。最终的准确率就是如下公式，其中 D 是数据集，$P^{AI} \in \mathbb{R}^{D\times 2}$ 是 AI 标注的，而 $p^H \in \mathbb{R}^D$ 是人类标注的：

$$z_{\text{acc}} = \frac{1}{D}\sum_{i=1}^{D} \mathbb{1}\left[\arg\max_j P_{i,j}^{AI} = p_i^H\right]$$

- Win Rate：给定两个策略生成的结果，人类选择更好的那个，然后统计胜率。
- Harmless Rate：人类认为 response 是 harmless 的比例，

另外，发现小的模型更容易出现 position bias，即给定两个候选，换一下顺序，模型还是觉得同一个位置的更好。缓解：每个 pair 反转前后各过一遍模型，然后得分 avg 一下。

2 O1

2.1 from r to Q*

这就是 OpenAI 神秘的 Q*？斯坦福：语言模型就是 Q 函数

From r to Q*: Your Language Model is Secretly a Q-Function

在 DPO 的基础上，引入 LLM 里的 token-level 的 MDP，用二分类的 preference feedback。发现了 3 个点：

- 尽管 DPO 是作为上下文多臂赌博机而派生出来的，但 DPO 模型的隐含奖励可在每个 token 层面上进行解释。
- DPO 模型的 likelihood search 类似在 decoding 阶段寻找一个 reward function。即在 token 层面的阐述方式下，经典的基于搜索的算法（比如 MCTS）等价于在 DPO 策略上的基于似然的搜索。
- 初始策略和参考分布的选择对于确定训练期间隐性奖励的轨迹非常重要。

OpenAI 秘密武器「草莓」计划曝光！Q* 推理能力大爆发，逼近 AGI L2 里程碑

5 级路线图：

- L1：聊天机器人，具有对话能力的 AI。
- L2：推理者，像人类一样能够解决问题的 AI。
- L3：智能体，不仅能思考，还可以采取行动的 AI 系统。
- L4：创新者，能够协助发明创造的 AI。
- L5：组织者，可以完成组织工作的 AI。

Strawberry 模型的目的是为了使公司的 AI 不仅能生成查询答案，还能提前计划，足够自主且可靠地浏览互联网，进行 OpenAI 所称的「深度研究」。

类似Star: Self-taught reasoner bootstrapping reasoning with reasoning能够通过迭代创建自己的训练数据，来「自我提升」到更高的智能水平。

Nature 封面：AI 训练 AI，越训越离谱

AI models collapse when trained on recursively generated data，对应下载的pdf

2.2 自我奖励

Self-Rewarding Language Models

「用 AI 训 AI」这事靠谱吗？

2.3 self-play

清华、北大等发布 Self-Play 强化学习最新综述

OpenAI o1 强化学习背后的自博弈（Self-play）方法介绍

万字长文推演 OpenAI o1 self-play RL 技术路线

有想入坑 RL-LLM 的同学吗？这个开源项目一个 GPU 够了，完成后欢迎来月之暗面 ~

https://github.com/inspirai/TimeChamber

2.4 Cursor

Scaling Law 瓶颈，Cursor 编程为什么这么强？团队参与新研究掏出秘密武器

Planning In Natural Language Improves LLM Search For Code Generation

2.5 Let’s verify step by step

o1 基石论文火爆传阅，Ilya 仍是关键先生！核心项目清北校友闪光

Let’s Verify Step by Step

OpenAI 使用过程监督提升数学推理能力

https://openai.com/index/improving-mathematical-reasoning-with-process-supervision/

2.6 O1 相关汇总

(toread)

OpenAI o1 要跟，怎么跟？这个 GitHub 项目把解读、博客、相关论文一网打尽

刚刚，OpenAI 震撼发布 o1 大模型！强化学习突破 LLM 推理极限

张俊林：OpenAI o1 的价值意义及强化学习的 Scaling Law

北大对齐团队独家解读：OpenAI o1 开启「后训练」时代强化学习新范式

Noam Brown 早已预示 o1 强大推理能力，演讲深度解析 AI 推理研究脉络

OpenAI o1 模型的前世今生 –toread

2.7 CoT 能让 Transformer 更强

谷歌再次痛失好局！OpenAI o1 证实谷歌 ICLR 2024 论文价值「四位华人贡献」

Chain of Thought Empowers Transformers to Solve Inherently Serial Problems

2.7.1 CoT：打破 Transformer 的 "并行诅咒"

传统的 Transformer 模型虽然在自然语言处理领域取得了巨大成功，但它有一个致命弱点：擅长并行计算，但不擅长串行推理。这就像一个超级聪明的孩子，能快速完成大量的计算题，但却无法理解简单的逻辑推理。

而 CoT (Chain of Thought，思维链) 技术的灵感来源于人类的思维过程，它可以让 Transformer 模拟人类的思考方式，通过生成一系列中间推理步骤，来解决那些需要逻辑推理的复杂问题。

2.7.2 CoT 的理论基础：从电路复杂度到 Transformer 表达能力

作者用电路复杂性理论来解释 CoT 的强大之处，将 Transformer 的计算过程与电路模型进行类比，并将 Transformer 能够解决的问题类别定义为 "CoT 复杂性类"

他们证明了传统的 Transformer 模型（没有 CoT）只能解决 AC0 电路能够解决的问题，而 AC0 电路是一种计算能力非常有限的电路模型。但是，如果加入 CoT，Transformer 的表达能力将得到质的飞跃！作者用数学严格证明了：

只要 CoT 步骤足够多，Transformer 就能模拟任意大小的布尔电路，从而解决 P/poly 问题，这是一个包含了 P 问题的更大的问题类别，相当于证明了 CoT 可以让 Transformer 解决几乎所有可以用计算机解决的问题。

2.7.3 CoT 的实验验证：从模加到电路值问题，CoT 全面胜出！

为了进一步验证 CoT 的有效性，论文作者设计了四个核心问题：

- 模加：计算两个数的和，并对某个整数取模
- 排列组合：计算一组排列的组合
- 迭代平方：对一个数进行多次平方运算
- 电路值问题：计算一个布尔电路的输出值

其中，模加问题可以用并行计算高效地解决，而其他三个问题则需要串行计算。

实验结果表明：

- 对于模加问题，即使不使用 CoT，Transformer 也能取得不错的效果
- 但对于其他三个问题，使用 CoT 可以显著提高 Transformer 的准确率，尤其是在模型深度较浅的情况下

2.7.4 讨论

CoT 能让模型推理能力无上限？田渊栋、LeCun 下场反对：两层 MLP 还能模拟全世界呢

Transformer 推理天花板被谷歌打破？DeepMind 首席科学家亮出 84 页 PPT，却遭 LeCun 反对

2.8 CoT or not

o1 带火的 CoT 到底行不行？新论文引发了论战

To CoT or not to CoT? Chain-of-thought helps mainly on math and symbolic reasoning

2.9 test-time scaling

小模型越级挑战 14 倍参数大模型，谷歌开启 Test-Time 端新的 Scaling Law

Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters

OpenAI o1 技术初探 1：整体框架，利用 Test-Time Scaling Law 提升逻辑推理能力—(写得比较细)

Scaling LLM Test-Time：谁说类 o1 推理一定要用 RL???

3B 模型长思考后击败 70B！HuggingFace 逆向出 o1 背后技术细节并开源

2.10 inference scaling

LLM Inference Scaling：姚班/OpenAI/CMU 8 月论文提前揭示 o1 核心原理

An Empirical Analysis of Compute-Optimal Inference for Problem-Solving with Language Models

2.11 扩散模型的 inference scaling

扩散模型也能推理时 Scaling，谢赛宁团队重磅研究可能带来文生图新范式

Inference-Time Scaling for Diffusion Models beyond Scaling Denoising Steps

2.12 O1 的评估

280 页 PDF，全方位评估 OpenAI o1，Leetcode 刷题准确率竟这么高

Evaluation of OpenAI o1: Opportunities and Challenges of AGI

2.13 LLM Reasoning

一文看懂 LLM 推理，UCL 汪军教授解读 OpenAI o1 的相关方法

首个 o1 复现开源 RL 框架 OpenR 来了，UCL、上交等高校联合团队发布

OpenR: An Open Source Framework for Advanced Reasoning with Large Language Models

A Tutorial on LLM Reasoning: Relevant methods behind ChatGPT o1

大模型不会推理，为什么也能有思路？有人把原理搞明白了

Procedural Knowledge in Pretraining Drives Reasoning in Large Language Models

2.14 开源模型 +O1

OpenAI o1 式思维链，开源模型也可以有，成功案例来了

2.15 复现 O1

技术上，如何复现 o1?

17 岁高中生写了个神级 Prompt，直接把 Claude 强化成了满血 o1

https://github.com/richards199999/Thinking-Claude/tree/main

OpenAI 最大秘密，竟被中国研究者破解？复旦等惊人揭秘 o1 路线图

Scaling of Search and Learning: A Roadmap to Reproduce o1 from Reinforcement Learning Perspective

2.16 O1 Replication Journey

上交大发布首个 OpenAI o1 复现项目进展报告，满满的经验洞察

上交大 o1 复现新突破：蒸馏超越原版，警示 AI 研发“捷径陷阱”

O1 Replication Journey: A Strategic Progress Report

https://github.com/GAIR-NLP/O1-Journey/blob/main/resource/report-part2.pdf

https://github.com/GAIR-NLP/O1-Journey

2.17 ScoRe

强化学习让大模型自动纠错，数学、编程性能暴涨，DeepMind 新作

OpenAI o1 技术初探 3：如何让模型拥有自我纠错的能力

2.18 LeCo

COLM 24 | 从正确中学习？大模型的自我纠正新视角

Learning From Correctness Without Prompting Makes LLM Efficient Reasoner

https://github.com/starrYYxuan/LeCo

2.19 Marco-O1

阿里推理模型来了！Marco-o1 发布即开源

阿里国际版 o1 来了，Marco-o1：聚焦开放式问题推理

Marco-o1: Towards Open Reasoning Models for Open-Ended Solutions

https://github.com/AIDC-AI/Marco-o1

2.20 OmniSearch

阿里多模态检索智能体，自带 o1 式思考过程！复杂问题逐步拆解，动态调整下一步检索策略

Benchmarking Multimodal Retrieval Augmented Generation with Dynamic VQA Dataset and Self-adaptive Planning Agent

https://github.com/Alibaba-NLP/OmniSearch

2.21 Coconut

田渊栋团队论文火了！连续思维链优于 CoT，打开 LLM 推理新范式

(toread)

Training Large Language Models to Reason in a Continuous Latent Space

一般而言，LLM 被限制在语言空间（language space）内进行推理，并通过思维链（CoT）来表达推理过程，从而解决复杂的推理问题。然而，语言空间可能并不总是最适合推理的。例如，很多单词 token 主要用于文本连贯性，而不是推理本身，而一些关键 token 则需要复杂的规划。

Coconut（连续思维链，Chain of Continuous Thought）不再通过语言模型头（language model head）和嵌入层将隐藏状态与语言 token 进行映射，而是直接将最后的隐藏状态（即连续思维）作为下一个 token 的输入嵌入。

2.22 O1 的安全机制

OpenAI 发布 49 页长文，讲述 o1 的安全机制

OpenAI o1 System Card

2.23 O1 架构

「七万字长文」从认知架构到实践部署：o1 与 o1 Pro 的系统性分析与内涵洞察 · 上篇

2.24 gemini 2.0 flash thinking

推理最强也最快，谷歌发布 Gemini 2.0 Flash Thinking，全面超越 o1-preview

https://ai.google.dev/gemini-api/docs/thinking-mode?hl=zh-cn

https://colab.research.google.com/github/google-gemini/cookbook/blob/main/gemini-2/thinking.ipynb

我自己复制的一个：https://colab.research.google.com/github/daiwk/llms_new/blob/main/gemini-2/thinking.ipynb

对应的 github：https://github.com/daiwk/llms_new/blob/main/gemini-2/thinking.ipynb

2.25 小模型的 O1

(toread)

让 7B 千问模型超越 o1，微软 rStar-Math 惊艳登场，网友盛赞

rStar-Math: Small LLMs Can Master Math Reasoning with Self-Evolved Deep Thinking

https://github.com/microsoft/rStar

2.26 meta-cot

迈向 System 2 推理，100 页论文硬核讲述 Meta-CoT

Towards System 2 Reasoning in LLMs: Learning How to Think With Meta Chain-of-Thought

2.27 sky-T1

(toread)

450 美元训练一个「o1-preview」? UC 伯克利开源 32B 推理模型 Sky-T1，AI 社区沸腾了

https://novasky-ai.github.io/posts/sky-t1/

https://huggingface.co/NovaSky-AI/Sky-T1-32B-Preview

2.28 其他的一些讨论

OpenAI o1 模型超全指南来了！ (使用技巧)

耗资 1.3 万，ASU 团队揭秘 o1 推理王者！碾压所有 LLM 成本超高，关键还会 PUA

LLMs Still Can't Plan; Can LRMs? A Preliminary Evaluation of OpenAI's o1 on PlanBench

大模型是否有推理能力? DeepMind 数月前的论文让 AI 社区吵起来了

Amortized Planning with Large-Scale Transformers: A Case Study on Chess

联手 OpenAI，吴恩达推出一门 o1 推理新课程，还免费

https://www.deeplearning.ai/short-courses/reasoning-with-o1/

4o-mini 只有 8B，o1 也才 300B！微软论文意外曝光 GPT 核心机密

- o1-preview 约 300B；o1-mini 约 100B
- GPT-4o 约 200B；GPT-4o-mini 约 8B
- Claude 3.5 Sonnet 2024-10-22 版本约 175B

3 DeepSeek R1

DeepSeek-R1 发布，性能对标 OpenAI o1 正式版

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning，自己转存了一份：pdf

arxiv 上：https://arxiv.org/pdf/2501.12948

https://huggingface.co/deepseek-ai/DeepSeek-R1，还有不少 distill 的，LM-studio 已经有了。。https://hf-mirror.com/lmstudio-community/DeepSeek-R1-Distill-Qwen-32B-GGUF

Jay Alammar：图解 DeepSeek-R1

3.1 背景

o1 提出了可以通过增加 cot reasoning process 的长度来进行 inference time scaling，尝试复现的方法有：

- process-based reward models：
  - Let's Verify Step by Step
  - Solving math word problems with process-and outcome-based feedback
  - Math-Shepherd: Verify and Reinforce LLMs Step-by-step without Human Annotations
- reinforcement learning：
  - Training language models to self-correct via reinforcement learning
- MCTS/beam search 的搜索方法：
  - Alphazero-like tree-search can guide large language model decoding and training
  - Solving olympiad geometry without human demonstrations
  - Deepseek-prover-v1.5: Harnessing proof assistant feedback for reinforcement learning and monte-carlo tree search

但这些方法都没有 o1 效果好，因此 R1 不借助监督数据，使用纯 RL(没有 SFT) 来完成 self-evolution，从而探索 LLM 在 reasoning 上的潜能。使用 DeepSeekV3 作为 base model，并使用Deepseekmath: Pushing the limits of mathematical reasoning in open language models的 GRPO 作为 RL 框架。

几 k 个 step 后，DeepSeek-R1-Zero 在一些 reasoning 的 benchmark 上取得了不错的效果，但仍然有可读性差、语言混合等问题，因此搞了 DeepSeek-R1，包括少量的冷启数据和一个多阶段的训练 pipeline。

- 收集数千的 cold-start 数据来 finetune DeepSeek-V3-Base 模型
- 执行类似 DeepSeek-R1-Zero 的面向 reasoning 的 RL 任务
- 在 RL 快收敛的时候，在 RL 的 checkpoint 上通过拒绝采样构建新的 SFT 数据，并和 DeepSeek-V3 的数据集（只选一些领域，如 writing、factual QA 和 self-cognition, 自我认知）进行结合，**重训** DeepSeek-V3-Base
- 用新数据 finetune 完后，再经过一个 RL 的过程，并考虑所有场景的 prompts==> 最终的模型

还搞了一些蒸馏模型，例如 Qwen2.5-32B 作为 base 模型，蒸馏 DeepSeek-R1 的效果比对它进行 RL 效果要好，说明大的 base model 发现的 reasoning patterns 对提升 reasoning 能力很关键。此外，发现 14B 的蒸馏模型比 QWQ-32B-preview 效果好很多，32B 和 70B 的蒸馏模型效果更好

3.2 DeepSeek-R1-Zero

3.2.1 RL 算法

GRPO（Group Relative Policy Optimization）：放弃了 Critic model，用 group scores 来替换。即对每个问题 q，从老的策略 $\pi_{\theta_{\text{old}}}$ 采样出一个 group 的输出 $\{o_1, o_2, \cdots, o_G\}$，再通过最大化如下目标来优化 policy model π_θ：

$$
\begin{aligned}
\mathcal{J}_{GRPO}(\theta) = \mathbb{E}\left[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{\text{old}}}(O \mid q)\right] \\
\frac{1}{G}\sum_{i=1}^{G}\left(\min\left(\frac{\pi_\theta(o_i \mid q)}{\pi_{\theta_{old}}(o_i \mid q)}A_i, \text{clip}\left(\frac{\pi_\theta(o_i \mid q)}{\pi_{\theta_{old}}(o_i \mid q)}, 1-\varepsilon, 1+\varepsilon\right)A_i\right) - \beta\mathbb{D}_{KL}\left(\pi_\theta || \pi_{ref}\right)\right),
\end{aligned}
$$

其中：

- $\mathbb{D}_{KL}\left(\pi_\theta \| \pi_{ref}\right) = \frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - \log\frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - 1$
- ε 和 β 是超参
- A_i 是 advantage，通过每个 group 的 reward 计算得来：

$$
A_i = \frac{r_i - \text{mean}\left(\{r_1, r_2, \cdots, r_G\}\right)}{\text{std}\left(\{r_1, r_2, \cdots, r_G\}\right)}
$$

3.2.2 Reward modeling

用的是 rule-based reward，包括如下两部分

- accuracy rewards：response 是否正确，例如对于一个数学问题，需要判断最终答案是否为给定的格式（specified format），并且能基于规则验证结果的正确性；leetcode 问题则可以用编译器来生成预先定好的测试用例的结果。
- format rewards：设计了一个 format reward model，来强制模型将其思考过程输出在 `<think>` 和 `</think>` 之间。

没有设计 neural reward model，因为：

- 发现在大规模的 RL 过程中，neural reward model 会受 reward hacking 的影响
- 重新训练 reward model 需要额外的训练资源，并且会让训练 pipeline 过于复杂

3.2.3 Training template

```
A conversation between User and Assistant. The user asks a question, and the Assistant solves it.
The assistant first thinks about the reasoning process in the mind and then provides the user
with the answer. The reasoning process and answer are enclosed within <think> </think> and
<answer> </answer> tags, respectively, i.e., <think> reasoning process here </think>
<answer> answer here </answer>. User: prompt. Assistant:
```

训练时使用如上的 template，故意将约束限制在这种结构格式上，避免任何特定内容的偏见（例如强制反射推理或促进特定的解决问题的策略），以确保能够在 RL 过程中准确观察模型的自然进展。

3.2.4 效果

指标含义：

- pass@1：**首次**生成答案时的成功率
- cons@64：majority vote (consensus) with 64 samples，模型在给出 **64 次生成尝试**中，是否能够**多次生成相同的正确答案**。(在这里提到了，来自文章：Self-consistency improves chain of thought reasoning in language model)

Model	AIME 2024		MATH-500	GPQA Diamond	LiveCode Bench	CodeForces
	pass@1	cons@64	pass@1	pass@1	pass@1	rating
OpenAI-o1-mini	63.6	80.0	90.0	60.0	53.8	1820
OpenAI-o1-0912	74.4	83.3	94.8	77.3	63.4	1843
DeepSeek-R1-Zero	71.0	86.7	95.9	73.3	50.0	1444

Table 2 | Comparison of DeepSeek-R1-Zero and OpenAI o1 models on reasoning-related benchmarks.

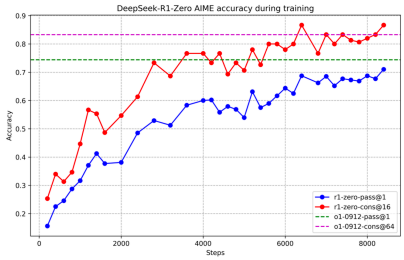


Figure 2 | AIME accuracy of DeepSeek-R1-Zero during training. For each question, we sample 16 responses and calculate the overall average accuracy to ensure a stable evaluation.

- 数学 (美国数学邀请赛)：
  - AIME2024：pass@1 上接近 o1-0912，cons@64 超了，均超越 o1-mini
  - MATH-500：pass@1 超越 o1-mini 和 o1-0912
- GPQA-Diamond（phd 级别的科学问题，包括化学、物理、生物）：pass@1 超越 o1-mini，不如 o1-0912
- 代码：
  - LiveCode Bench：不如 o1-0912 和 o1-mini
  - codeforces 的 rating：不如 o1-0912 和 o1-mini

另外，在 AIME2024 上，随着训练，pass@1 能从十几涨到 70 几。

3.2.5 Self-evolution Process

- 随着训练的进行，生成的 response 长度也在变长，说明思考时间也变长了，其中的 reasoning tokens 从数百涨到了数千。
- 随着测试时间的增加，**涌现出了复杂的行为**，即不由自主地 (spontaneously) 出现了（这些行为并非预先在代码里设计的）：
  - 反思：重新看并且评估 (revist and reevaluate) 之前的 steps
  - 探索：尝试其他方法来解决问题

3.2.6 Aha Moment

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a-\sqrt{a+x}} = x$ is equal to
Response: <think> To solve the equation $\sqrt{a-\sqrt{a+x}} = x$, let's start by squaring both $\cdots$ $\left(\sqrt{a-\sqrt{a+x}}\right)^2 = x^2 \implies a-\sqrt{a+x} = x^2$. Rearrange to isolate the inner square root term: $(a-x^2)^2 = a+x \implies a^2-2ax^2+(x^2)^2 = a+x \implies x^4-2ax^2-x+(a^2-a) = 0$ $\ldots$ Wait, wait. Wait. That's an aha moment I can flag here. Let's reevaluate this step-by-step to identify if the correct sum can be $\cdots$ We started with the equation: $\sqrt{a-\sqrt{a+x}} = x$ First, let's square both sides: $a-\sqrt{a+x} = x^2 \implies \sqrt{a+x} = a-x^2$ Next, I could square both sides again, treating the equation: $\cdots$ $\ldots$

Table 3 | An interesting "aha moment" of an intermediate version of DeepSeek-R1-Zero. The model learns to rethink using an anthropomorphic tone. This is also an aha moment for us, allowing us to witness the power and beauty of reinforcement learning.

在某一个中间版本，模型突然输出上面的 aha-moment，可以看到不需要显式地教模型如何解决，只要给予正确的激励（incentives），模型会自己开发出进阶的问题解决策略，这也是 RL 的力量和美。

3.3 DeepSeek-R1

2 个目标：

- 用一小部分高质量数据来做冷启能否提升推理效果，或者加速收敛?
- 如何训练一个 user-friendly 的模型，既能提供准确和连贯的 CoT，又有很强的通用能力?

3.3.1 阶段 1：Cold Start

从 base model 开始 RL 训练时，为了防止冷启阶段训练的不稳定，构建了少量（数千）的长 CoT 数据来 finetune 模型，作为初始的 RL actor。试了几种方法

- few shot prompting，用一个 long CoT 作为例子
- 直接在 prompt 里要求模型生成详细的答案，并加上反思和验证
- 收集 deepseek-r1-zero 的输出，并找人类标注员后处理进行修改

相比 zero，冷启数据有如下好处：

- 可读性：zero 的可读性不太好，例如语言混合、缺少 markdown 高亮。因此设计了一个可读性比较好的 pattern，即 `|special_token|<reasoning_process>|special_token|<summary>`，并把没有 summary 的结果扔掉
- 潜力：依据人类先验精心设计的 pattern 产出的冷启数据，取得了比 zero 更好的效果，故迭代地训练对于 reasoning models 应该是一种更好的方式

3.3.2 阶段 2：Reasoning-oriented Reinforcement Learning

在冷启数据上 tune 完之后，和 r1-zero 一样，进行大规模的 RL 训练（在 reasoning-intensive 任务的数据集上训的?）。此外，为了解决 CoT 里语言混合的问题，加了一个 **language consistency reward**（计算目标语言在 CoT 中的比例），直接和原来的 reward 相加。虽然消融实验显示这种 alignment 会让效果略微下降，但这更符合人类偏好，可读性更强。

3.3.3 阶段 3：Rejection Sampling and Supervised Fine-Tuning

用阶段 2 的 ckpt，然后在 writing, role-playing, and other general-purpose 这类任务上构建 SFT 数据集，在约 80w 的样本上对 DeepSeek-V3-Base SFT 了 2 个 epoch。

- reasoning data：在前一个 RL 阶段，只用那些能用 rule-based rewards 来衡量的数据。
  - 这个阶段加入了更多的数据，有一些是使用**生成式的 reward model**，即把 ground truth 和模型预测结果输入给 DeepSeek-V3 做 judgement。
  - 因为模型输出有时会很混乱或者无可读性，所以删掉一些满足这些条件的 CoT：混合语言、长的段落、代码段
  - 对每个 prompt，采样多个 response，并只保留一个正确的。最终收集了大概 60w 的 reasoning 相关的训练样本
- non-reasoning data：
  - 采用 DeepSeek-V3 的 pipeline，并复用部分 DeepSeek-V3 的 SFT 数据集。
  - 使用 DeepSeek-V3 时，在 prompt 里提到在回答答案前要生成潜在的 CoT；对于简单的如 hello 的问题，就不用输出 CoT。
  - 最终收集了大概 20w 的 non-reasoning 训练样本

3.3.4 阶段 4：Reinforcement Learning for all Scenarios

将 reward 信号和多样化的数据分布相结合，提升 helpfulness and harmlessness，同时 refine reasoning 能力，

- reasoning 数据：用 R1-Zero 的 RL，使用 rule-based reward

- 通用数据：用 V3 的 pipeline，还是用 reward model，用以学习复杂和细分领域的人类偏好

另外，提升如下两方面的能力：

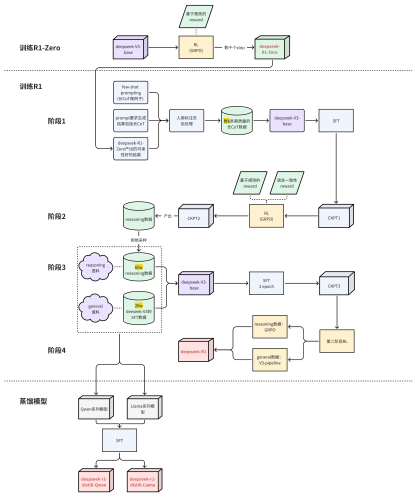
- helpfulness：仅关注最终的 summary，保证评估强调的是 response 的有用性和相关性，而尽量少地干涉潜在的 reasoning 过程
- harmlessness：同时评估 reasoning 过程和 summary，尽量缓解在生成的过程中出现的风险、偏见和有害内容

3.4 蒸馏

基于 Qwen2.5-Math-1.5B、Qwen2.5-Math-7B、Qwen2.5-14B、Qwen2.5-32B、Llama-3.1-8B、Llama-3.3-70B-Instruct 蒸馏，用的数据集是 deepseek-R1 的第二阶段 RL 使用的 80w 数据，只进行了 SFT，没有 RL。理论上 RL 应该会效果更好，留给其他研究者去搞了。

[https://medium.com/@prabhudev.guntur/how-to-distill-deepseek-r1-a-comprehensive-guide-c8ba04e2c28c](https://medium.com/@prabhud
to-distill-deepseek-r1-a-comprehensive-guide-c8ba04e2c28c) 可以参考这个自己蒸馏一个

3.5 小结



对应的流程图文档

3.6 使用注意

3.6.1 deepseek-r1 使用指南

刚刚，DeepSeek 官方发布 R1 模型推荐设置，这才是正确用法

- temperature 设置在 0.5-0.7 之间，推荐 0.6，以防止无休止的重复或不连贯的输出。
- 不要 system prompt，直接放到 user prompt 里
- 数学问题，建议 prompt 里直接 `Please reason step by step, and put your final answer within \boxed{}.`
- 有些时候模型会自己跳过 think 的过程，如果要强制输出，可以强制模型在 output 前输出 `<think>\n`

上传文件的 prompt：

```
file_template = \
"""[file name]: {file_name}
[file content begin]
{file_content}
[file content end]
{question}"""
```

联网的 prompt：

```
search_answer_zh_template = \
'''# 以下内容是基于用户发送的消息的搜索结果:
{search_results}
在我给你的搜索结果中，每个结果都是[webpage X begin]...[webpage X end]格式的，X代表每篇文章的数字索引。请
在回答时，请注意以下几点：
- 今天是{cur_date}。
- 并非搜索结果的所有内容都与用户的问题密切相关，你需要结合问题，对搜索结果进行甄别、筛选。
- 对于列举类的问题（如列举所有航班信息），尽量将答案控制在10个要点以内，并告诉用户可以查看搜索来源、获得
- 对于创作类的问题（如写论文），请务必在正文的段落中引用对应的参考编号，例如[citation:3][citation:5]，不
- 如果回答很长，请尽量结构化、分段落总结。如果需要分点作答，尽量控制在5个点以内，并合并相关的内容。
- 对于客观类的问答，如果问题的答案非常简短，可以适当补充一到两句相关信息，以丰富内容。
- 你需要根据用户要求和回答内容选择合适、美观的回答格式，确保可读性强。
- 你的回答应该综合多个相关网页来回答，不能重复引用一个网页。
- 除非用户要求，否则你回答的语言需要和用户提问的语言保持一致。

# 用户消息为：
{question}'''
```

3.6.2 openai 推理使用指南

刚刚，DeepSeek 揭秘 R1 官方同款部署设置，温度 =0.6！OpenAI 推理指南同时上线

https://platform.openai.com/docs/guides/reasoning-best-practices

简单理解：

- 推理模型就像一位经验丰富的高级同事———你只需告诉他们最终目标，就能相信他们自主完成所有细节工作。
- GPT 模型则更像一位新手同事———你需要提供明确详细的指示，才能让他们准确完成特定的输出任务。

各自特点：

- 速度和成本：选择 GPT 模型，因为它们处理速度更快，成本更低
- 执行明确任务：选择 GPT 模型，它们在处理界定清晰的任务时表现出色
- 准确性和可靠性：选择 o 系列模型，它们是可靠的决策专家
- 复杂问题解决：选择 o 系列模型，它们善于处理模糊和复杂的问题

推理模型的优势：

- 处理模糊任务：推理模型特别擅长处理信息有限或零散的情况，只需通过简单的提示词就能理解用户意图并妥善处理指令中的信息缺口。值得注意的是，推理模型通常会在做出未经验证的猜测或填补信息空缺之前，主动提出澄清性问题。
- 大海捞针：当需要处理大量非结构化信息时，推理模型特别擅长理解内容并精准提取出回答问题所需的关键信息。
- 在大型数据集中发现关系和细微差别：
  - 推理模型特别擅长分析包含数百页密集、非结构化信息的复杂文档，如法律合同、财务报表和保险索赔等。这些模型在识别文档之间的关联性，并基于数据中隐含的事实做出决策方面，表现尤为突出。
  - 推理模型还特别擅长理解细微的政策和规则，并将其准确应用于具体任务中以得出合理结论。
- 多步骤 AI 智能体规划：推理模型在 AI 智能体规划和策略制定中发挥着关键作用。将推理模型作为「计划者」时效果显著：它能为问题制定详细的多步骤解决方案，并根据具体需求（高智能或低延迟）选择和分配合适的 GPT 模型（执行者）来完成各个步骤。
- 视觉推理能力：截至目前，o1 是唯一一个具备视觉处理能力的推理模型。与 GPT-4o 相比，o1 的独特优势在于它能够准确理解最具挑战性的视觉内容，包括结构不规则的图表和表格，以及质量欠佳的图片。
- 代码审查、调试和质量改进：推理模型在审查和改进大规模代码方面表现突出。考虑到这类模型的较高延迟特性，通常将代码审查任务安排在后台运行。虽然 GPT-4o 和 GPT-4o mini 凭借较低的延迟可能更适合直接编写代码，但在那些对延迟要求相对不那么严格的代码生成场景中，o3-mini 表现同样出色。
- 评估和基准测试其他模型的响应：推理模型在对其他模型的输出进行基准测试和评估方面表现优异。数据验证对确保数据集的质量和可靠性至关重要，这一点在医疗保健等敏感领域尤其重要。

prompt 注意点：

- 用开发者消息取代系统消息：自 o1-2024-12-17 版本起，推理模型开始支持开发者消息（developer message）而非系统消息（system message）。

- 保持提示词简洁明确：推理模型最擅长理解和响应简短、清晰的指令。
- 避免使用 CoT 提示：由于模型内置推理能力，因此无需特别提示它们「一步一步思考」或「解释推理过程」。
- 善用分隔符增强清晰度：使用 Markdown、XML 标签和章节标题等分隔符来明确区分输入的不同部分，这有助于模型准确理解各个章节的内容。
- 优先尝试零样本学习：推理模型通常无需少样本示例即可产出优质结果，因此建议先尝试不含示例的提示词。如果对输出结果有更复杂的要求，再考虑在提示词中添加输入和期望输出的示例。请注意确保示例与提示词指令严格匹配，因为不一致可能导致性能下降。
- 提供明确约束条件：如果需要对模型的响应施加具体限制（例如「提供预算控制在 500 美元以内的解决方案」），请在提示词中明确列出这些约束条件。
- 明确定义目标：在指令中，请详细说明判定响应成功的具体参数，并引导模型持续优化推理过程，直到达成设定的成功标准。
- Markdown 格式说明：从 o1-2024-12-17 版本开始，API 中的推理模型默认不会生成带有 Markdown 格式的响应。如果确实需要在响应中包含 Markdown 格式，请在开发者消息的首行添加「Formatting re-enabled」字符串。

4 R1 的部署

完整的 671B MoE DeepSeek R1 怎么塞进本地化部署？详尽教程大放送！

https://github.com/kvcache-ai/ktransformers

5 R1 的讨论

华人研究团队揭秘：DeepSeek-R1-Zero 或许并不存在「顿悟时刻」

为什么说 DeepSeek 的 R1-Zero 比 R1 更值得关注？

LeCun 痛批硅谷傲慢病！圈内爆火长文：DeepSeek R1-Zero 比 R1 更重要，成 AGI 破局关键

DeepSeek 用的 GRPO 占用大量内存？有人给出了些破解方法，对应：https://github.com/huggingface/trl/issues/2709

刘知远硬核解读 DeepSeek：大模型强化学习技术原理与大模型技术发展研判

大神卡帕西拿 DeepSeek R1 讲强化学习！最新大模型内部机制视频爆火，“没有技术背景也能看懂”

对应代码：https://github.com/EurekaLabsAI

陈巍：DeepSeek 是否有国运级的创新？2 万字解读与硬核分析 DeepSeek V3/R1 的架构

Sebastian Raschka：关于 DeepSeek R1 和推理模型，我有几点看法

4500 美元复刻 DeepSeek 神话，1.5B 战胜 o1-preview 只用 RL！训练细节全公开

从想太多到想不透？DeepSeek-R1 等长推理模型也存在「思考不足」问题

6 R1 前后的一些工作

6.1 S1

s1：Simple test-time scaling

https://github.com/simplescaling/s1

训练 1000 样本就能超越 o1，李飞飞等人画出 AI 扩展新曲线

（从 xhs 上抄的）有几点思维误区：

- s1 超过的是 o1-preview，没有超过 o1 甚至 o1-mini，有很大区别
- s1 的效果不敌 deepseek-r1 800k 数据蒸馏的 32B 模型，差了不少，不是媲美
- s1 即使使用全量 59k 数据的效果也没有提高很多，甚至在 math 上还有下降，所以核心是数据质量
- 1k 数据是从 59K 数据中筛选出来的，不是直接有 1K 数据就可以
- s1 使用 1k 数据是节省了训练时间，但蒸馏的难点在蒸馏数据的构造上

6.2 LIMO

817 样本激发 7 倍推理性能：上交大「少即是多」定律挑战 RL Scaling 范式

LIMO: Less is More for Reasoning

https://github.com/GAIR-NLP/LIMO

6.3 PRIME

挑战 DeepSeek-R1-Zero！PRIME：仅 10% 数据刷新数学推理 SOTA，隐式奖励技术颠覆 RL 训练

Process Reinforcement through Implicit Rewards

6.4 TPO

推理时也能做偏好优化，无需额外重训练，来自上海 AI Lab 港中文等

Test-Time Preference Optimization: On-the-Fly Alignment via Iterative Textual Feedback

https://github.com/yafuly/TPO

6.5 Huginn

推理模型新路线开源！与 DeepSeek 截然不同，抛弃思维链不用人类语言思考

Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach

https://github.com/seal-rg/recurrent-pretraining

抛弃长思维链和人类的语言，直接在连续的高维潜空间用隐藏状态推理，可自适应地花费更多计算来思考更长时间。

6.6 Goedel-Prover

哥德尔-Prover 超过 DeepSeek-Prover，金驰、陈丹琦团队造出当前最强形式化推理模型

Goedel-Prover: A Frontier Model for Open-Source Automated Theorem Proving

https://github.com/Goedel-LM/Goedel-Prover

6.7 Kimi K1.5

追平满血版 o1 的国产多模态模型终于来了！训练细节全部公开

https://github.com/MoonshotAI/kimi-k1.5

Kimi k1.5: Scaling Reinforcement Learning with LLMs，自己转存了一份：pdf

6.8 O1 Embedder

O1 Embedder: Let Retrievers Think Before Action

生成关于输入查询的 thought，然后和 question 一起拼接，然后分别独自生成嵌入，然后池化聚合。也就是说，这个 Embedding 模型比之前的模型多了个 thought 的输出。

微调，两个并行任务，一个是 thought 生成，一个是对比学习。

微调数据怎么来，那就是生成后进行打分评价。首先，使用 LLM 生成初始思想，然后使用检索评分器来根据初始思想和目标文档之间的相关性评分，最终通过多数投票选择最佳 thought。

6.9 小结

从 ReFT, Kimi K1.5 到 DeepSeek R1，聊聊 Reasoning Model 的精巧实现

张俊林：MCST 树搜索会是复刻 OpenAI O1/O3 的有效方法吗

7 R1 的复现

7.1 open-r1

https://github.com/huggingface/open-r1/

开源 22 万条 DeepSeek R1 的高质量数据！你也能复现 DeepSeek 了

https://huggingface.co/datasets/open-r1/OpenR1-Math-220k

7.2 unlock-deepseek

DeepSeek R1 Zero 中文复现教程来了！

7.3 unsloth

7.4 tinyzero

8 O3

刚刚，OpenAI 放出最后大惊喜 o3，高计算模式每任务花费数千美元

8.1 Monkey

o3 并非独门秘技，谷歌已发背后关键机制，方法更简单、成本更低

Large Language Monkeys: Scaling Inference Compute with Repeated Sampling